

VERITYGATE: A Four-Gate Schema-Level Faithfulness Framework and Paired Benchmark for Grounded LLM Narrations over Structured Evidence

Sachin Gupta

Independent Researcher

San Jose, California, USA

sachinkg12@gmail.com

Abstract

Fluent LLM explanations may not follow the evidence from a structured system. We present VERITYGATE, a four-gate checker for declared evidence IDs, entities, numbers, and claim types. It checks a fixed schema; it does not verify every fact in the prose. At $r=0$ and $r=1$, we test 900 instances per setting (450 grounded–ungrounded pairs) with GPT-4o-mini, Llama-3.3-70B, and Claude Sonnet 4.6. Under this schema-level contract and before repair, 80.3% of mini claims and 47.9% of Sonnet claims fail. These are verifier rejection rates, not prose-hallucination rates. One repair pass raises claim survival from 19.7% to 28.0% for mini and from 52.1% to 54.3% for Sonnet. Verified claims per example change by +0.14 for mini, -0.71 for Llama, and -0.47 for Sonnet, so survival and output volume must be reported together. A second Sonnet pass gives no clear gain. At $r=1$, Gate 4 covers 97.0%, 98.7%, and 100% of failing claims for mini, Llama, and Sonnet. Small human studies support the rules but show gaps between schema checks and correct prose. A domain-specific GPT-4o judge test shows an order effect, so it is only a usefulness check. We release the code and data.

1 Introduction

The way users interact with structured decision systems is changing. A user asks for a credit card recommendation, a treatment suggestion, or a loan risk assessment. A deterministic backend (a solver, a rules engine, a model) computes the answer. Then a large language model writes the explanation. This two-stage shape is increasingly common in tool-augmented and retrieval-augmented production systems for delivering grounded explanations of structured outputs.

The convenience hides a problem. The LLM, even when given the full evidence graph from the backend, still makes things up. It cites cards that

do not exist. It rounds numbers in ways that change their meaning. It flips signs. It invents facts that sound plausible.

In production, this is not a research curiosity. In *Moffatt v. Air Canada* (2024 BCCRT 149) ([British Columbia Civil Resolution Tribunal, 2024](#)), the airline was held legally liable for false bereavement-fare information its support chatbot gave to a customer. In March 2024, New York City’s MyCity chatbot (a Microsoft-powered system for small business guidance) was found advising businesses to take a cut of workers’ tips and lock out tenants, both illegal ([Lecher, 2024](#)). In February 2025, three attorneys at a top-50 U.S. law firm were sanctioned for filing a motion in which eight of nine cited cases were AI-fabricated ([Pazanowski, 2025](#)). As of mid-2026, a public database of court rulings on AI-hallucinated filings has documented over 1,400 decisions ([Charlotin, 2025](#)). The cost of a hallucination in a regulated sector is no longer hypothetical.

Existing defenses are not enough. Safety alignment within the model is trained against user-facing harms: jailbreaks, harmful content, direct prompt injection. It is not trained against “cite only the data I gave you.” Constrained decoding methods like Outlines and JSON schema mode force syntactic structure but do not check semantic correctness against an external backend. LLM-as-judge approaches can suffer position effects (our single judge chooses slot A in 83% of calls) and, in our diagnostic, prefer fluent ungrounded narrations over verifier-emitted ones, despite having no access to the evidence graph.

We present VERITYGATE, a faithfulness evaluation framework for grounded LLM narrations, with three contributions:

1. **A claim-level faithfulness taxonomy** for grading LLM narrations against a structured backend’s evidence: evidence existence, entity allowlisting, number binding, and claim-

type rule satisfaction. The taxonomy and verifier interface are designed to be domain-portable and apply to any pipeline where an LLM narrates over a structured backend that emits typed evidence with allowed entities and numbers; the evidence-graph builder and claim-type rules are domain-specific.

2. **A paired benchmark** (900 instances at each main setting, $r=0$ and $r=1$: 50 profiles $\times$ 3 goals $\times$ 3 narrators = 450 matched settings; grounded and ungrounded variants yield 900 instances) with pre-registered co-primary metrics and paired bootstrap CIs on every contrast. The benchmark enables head-to-head comparison of LLM backends and repair strategies under a single schema-level structural faithfulness metric (*claim-survival rate*).
3. **An inference-time repair pipeline** that returns failing claims to the LLM with specific verifier errors for a single repair pass. No modification to the LLM or the backend. A second pass shows no further detectable gain at $r=2$ on Sonnet, an empirical finding consistent with prior self-correction work.

We additionally contribute a methodology lesson from a counterbalanced LLM-as-judge protocol with an observed 83% position-A rate and 0% tie rate on one frontier judge: prose-only preference is not a substitute for mechanical faithfulness verification (Section 4.3).

We evaluate the pipeline on credit card portfolio optimization as a case study. The deterministic backend is a mixed-integer linear program (MILP) that computes the optimal 1–3 card portfolio for a user spending profile; the LLM narrator writes the explanation; VERITYGATE sits between them. We choose MILP as the concrete instantiation because it produces structured evidence with clean, audit-quality ground truth; the framework requires only that the backend emit typed evidence with allowed entities and numbers, a contract we revisit in Section 5.¹

2 A Four-Gate Verification Taxonomy

We catalog the failure modes of LLM narrators when given full structured grounding. At each

of $r=0$ and $r=1$, 900 narrations form 450 paired (grounded, ungrounded) comparisons across three LLM backends; four families account for essentially all observed failures. Each family corresponds to one verification gate.

Gate 1: Evidence Existence. Every cited evidence identifier must exist in the structured backend output. LLMs frequently fabricate identifiers that sound right but do not appear anywhere in the input. At $r=0$ (no repair) Sonnet 4.6 produces 1 evidence-existence error per 150 grounded instances and GPT-4O-MINI produces 6; after one repair pass these drop to 0 and 4 respectively (Table 2). The check is deterministic: a set membership against the backend’s emitted identifier set.

Gate 2: Entity Allowlisting. Cited entity names (card identifiers, category names, currency codes) must come from the deployment’s allowed entity set. LLMs paraphrase entity names from training prior, writing “Amex Blue Cash” when the backend used `amex-bcp`, or inventing “us-bank-altitude-go” when no such card was selected. The check is also deterministic: set membership against the backend’s allowed-entity list.

Gate 3: Number Binding. Cited number strings (dollar amounts, percentages, counts) must exactly match a canonical string surfaced from the structured backend. LLMs round, sign-flip, fabricate, or change decimal formatting. At $r=0$ GPT-4O-MINI produces 388 number errors across 150 grounded instances and Llama-70B produces 182; Sonnet produces 0 at $r=0$ and 1 at $r=1$. These counts include exact-format mismatches such as `2400.0` versus `2400.00`. Small integers (0–3) are allowed by convention. The evaluated check is exact string membership.

Gate 4: Claim-Type Rules. Claims of a given type must cite evidence of the required type. A comparison claim must cite a winner-by-category evidence node; a fee-justification claim must cite a fee-break-even node; a cap-switch claim must cite cap-hit evidence and, when those node types are present in the graph, both cap-hit and allocation-segment evidence. LLMs frequently produce structurally well-formed claims that cite the wrong evidence category. This is the hardest gate in our evaluation: on Sonnet 4.6 the count drops only from 985 ($r=0$) to 841 ($r=1$) across 150 instances, and a second repair pass moves it no further (Section 4). Across these three models and repair set-

¹The system’s name reflects its function: four mechanical gates (one for each failure mode in the taxonomy) through which every generated claim must pass before it ships to the user.

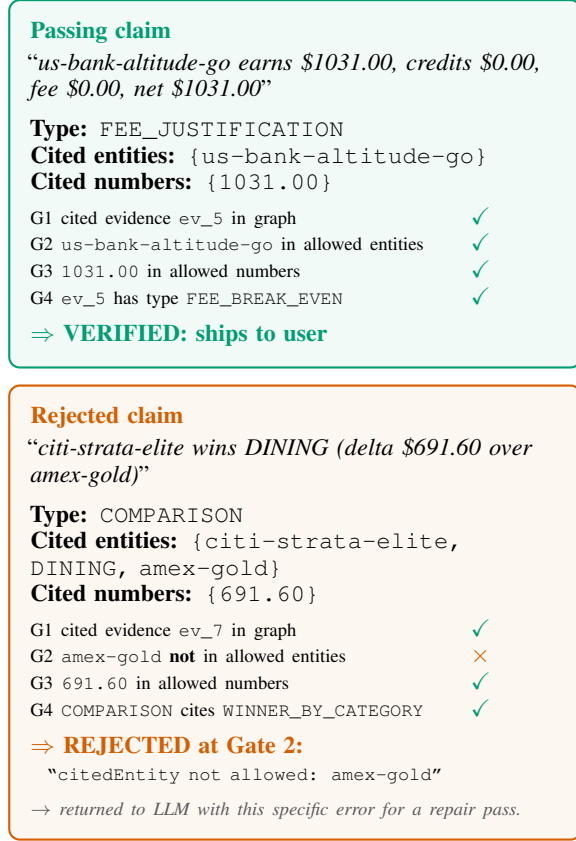


Figure 1: Illustrative traces reconstructed from claim text observed in our paper-grade evaluation; identifiers and structured declarations are simplified to isolate one gate at a time. *Top:* a FEE_JUSTIFICATION claim passes all four gates and ships. *Bottom:* a COMPARISON claim fails Gate 2 (the cited entity amex-gold is not in the allowed-entity list), is rejected with an explicit error message, and is returned to the LLM for a single repair pass. These are pedagogical traces rather than exact serialized log records. This is the verification contract that mechanically enforces structural faithfulness at the schema level.

tings, this persistence suggests a type-routing bottleneck; broader model and domain studies are needed.

Figure 1 shows two illustrative traces reconstructed from claim text observed in our paper-grade evaluation, making the four-gate contract concrete before we describe the framework architecture (Section 3).

Table 1 summarizes the four gates and their empirical profiles.

3 The VERITYGATE Framework

VERITYGATE operates as a transparent layer between the deterministic backend and the LLM narrator. It requires no modification to either side.

The backend emits structured evidence in its native format; the LLM is called with a standard prompt. The framework performs three-stage validation.

3.1 Stage 1: Evidence Graph Construction

We build an evidence graph from the backend output. Each evidence block becomes a node; edges connect logically related nodes (for example, a cap-hit node depends on the winner-by-category node for the same category). The graph also carries two allowlists:

- **Allowed entities:** all entity strings that appear in the backend output (card identifiers, category names, currency codes).
- **Allowed numbers:** all numbers that appear in any evidence node’s structured fields, plus a small-integer allowlist (0, 1, 2, 3).

The graph carries a SHA-256 digest of its canonical form. This makes the verification step deterministic and auditable: any two runs over the same backend output produce the same digest.

3.2 Stage 2: Four-Gate Verification

The narrator LLM is prompted (strict JSON) to emit prose plus a parallel list of structured claim objects with declared `citedEvidenceIds`, `citedEntities`, and `citedNumbers` (schema and Gate 4 type rules in Appendix C; prompt templates in Appendix D). The four gates check these declared structured fields against the evidence graph; the **guarantee** is *schema-level structural correctness*, not prose-level faithfulness: an under-citing LLM can mention unverified entities or numbers in prose that the verifier never inspects (Appendix A).

Each claim is checked against the four gates from Section 2. The check is purely structural: no LLM is involved. If any claim fails any gate, the claim is rejected. Crucially, rejection is independent of fluency. A grammatically beautiful, plausible-sounding claim that cites a non-existent number is rejected just like one containing an unallowed declared value.

The output of Stage 2 is two lists: *verified claims* (passed all four gates) and *failing claims* (with attached error messages indicating which gate(s) failed and which strings were unallowed).

Gate	What it checks	Detection	Empirical grounded error range (150 inst. per cell)
1. Evidence Existence	Cited evidence id is in the graph	Set membership	$r=0$: 1–6 $r=1$: 0–4 (Sonnet best, mini worst)
2. Entity Allowlisting	Cited entity is in allowed set	Set membership	$r=0$: 0–47 $r=1$: 0–43 (Sonnet best, mini worst)
3. Number Binding	Cited number string is in allowed set	Exact string membership	$r=0$: 0–388 $r=1$: 1–97 (Sonnet best, mini worst)
4. Claim-Type Rules	Claim type cites required evidence type	Type-rule check	$r=0$: 885–985 $r=1$: 652–841 (all tiers high)

Table 1: The four verification gates. Gates 1–3 are largely addressed by allowlist surfacing plus one repair pass on the frontier model. Gate 4 is the persistent residual mode in the evaluated cells: every tier produces hundreds of type-rule errors after repair, and a second Sonnet repair pass does not significantly reduce them (Section 4).

3.3 Stage 3: Repair Loop and Permissive Emission

If any claim failed verification, the framework can optionally re-prompt the LLM with the original context plus the specific verifier errors (“Your previous claims failed for these reasons. Use only the allowed strings.”). The model produces a new claim list, which is verified again. We keep whichever of the initial and post-repair lists has fewer failing claims (intra-pass selection; distinct from the deployment-time best-of- $(r=0, r=1)$ policy in Appendix E).

We use one repair pass as the headline operating point. On Sonnet, a second pass produces a paired Δ that is not statistically distinguishable from zero (Section 4). For Llama, one pass reduces shipped volume without a detectable survival gain, so deployments should select the operating point by model and report both co-primary metrics.

The final output is the *verified claim subset*: only claims that pass all four gates ship to the user, as their concatenated `text` fields. We call this *permissive emission*: emit whatever passes, drop what fails. The user never sees a failing claim. Permissive emission is a *filtering* policy, not a generation-time correction: the underlying LLM continues to fabricate; claim-survival measures what passes verification (what the user sees), not generator improvement.

Figure 2 shows the overall flow.

4 Evaluation

We evaluate VERITYGATE along three dimensions: claim-survival rate (the share of parsed-and-retained claims that pass verification), shipped verified claims per instance (the user-visible output), and judge-based usefulness preference (a secondary metric).

4.1 Setup

Case study and profiles. We test on credit card portfolio optimization. The deterministic backend is a MILP that emits six task-facing evidence types (cap-hit, winner-by-category, allocation-segment, fee-break-even, earn-rate, assumption); the schema also permits four support/container types. It is independently validated on 1,000 real households drawn from the U.S. BLS Public-Use Microdata (U.S. Bureau of Labor Statistics, 2024) (Spearman $\rho = 0.896$ vs ground-truth optimal portfolios across 25 survey waves). We rely on that validation for the credibility of the structured ground truth and do not re-derive it here. For this evaluation we use 50 parametric profiles $\times$ 3 goal types (cashback, flexible points, program points) = 150 (profile, goal) instances, fixed seed 42, stratified to include cap-boundary cases. Each of $r=0$ and $r=1$ has 150×3 narrators $\times$ 2 variants = **900 benchmark instances**, forming 450 matched grounded–ungrounded pairs. We add a Sonnet-only ablation at $r=2$ (300 more). The released result records contain 16,180 parsed-and-retained claim objects across these conditions. The verifier-passing subset is what we report as “shipped” in Table 2.

LLM backends. Three narrators spanning the capability range: GPT-4O-MINI-2024-07-18 (OpenAI, small closed), LLAMA-3.3-70B-INSTRUCTURBO (Meta, via Together AI, open-weights mid), and CLAUDE SONNET 4.6 (Anthropic, frontier closed). The judge model is GPT-4O-2024-08-06: a different model from the smallest narrator (GPT-4O-MINI) within the same vendor, and a different family entirely from the frontier narrator.²

²All API calls used temperature 0.0 with no provider-side caching; calls were made between 2026-03-01 and 2026-05-14. Cost estimates (Section 4.4) use vendor list pricing as of 2026-05-14, with per-call token counts measured from API

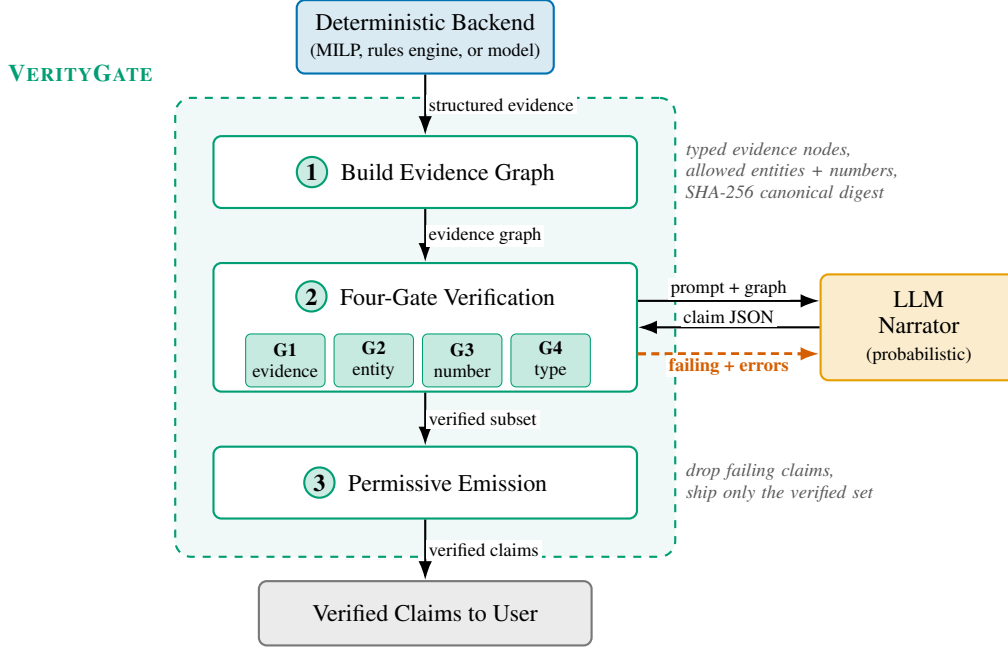


Figure 2: Three-stage interception architecture of VERITYGATE, operating as a transparent layer between the deterministic backend (here a MILP optimizer) and the LLM narrator. Stage 1 constructs the evidence graph with typed nodes, an allowed-entity set, and an allowed-number set. Stage 2 verifies each parsed claim against four gates (G1: evidence existence, G2: entity allowlist, G3: number binding, G4: claim-type rules); the optional repair loop (vermillion dashed arrow) re-prompts the LLM with the verifier errors for one pass. Stage 3 is a pure filter: only claims passing all four gates ship.

Variants. For each (profile, goal, model) we run two paired variants. *Grounded*: prompt includes the full structured evidence, the allowed entity list, and the allowed number list. *Ungrounded*: prompt includes only the goal name, the portfolio card list, and the top-line breakdown (matching the engine’s production `UngroundedLlmNarrator`).

Two runs, plus an ablation. We perform two paired runs: $r=0$ (no repair, baseline) and $r=1$ (one repair pass, our headline configuration). We also run a Sonnet-only ablation at $r=2$ to confirm diminishing returns.

Pre-registration. Co-primary metrics, pairing units, statistics, and explicit non-claims were pre-registered; see Appendix E. The protocol called the survival denominator “generated claims.” The evaluation used a lenient parser that skipped malformed individual records and marked an instance `schemaFailed` only if no record remained. Thus `totalClaims` counts parsed-and-retained claims; a fully failed instance ships zero but adds zero claims to this denominator. Pre-parse arrays were not archived, so the partial-record drop rate cannot be recovered.

responses.

Statistical procedure. All confidence intervals use cluster bootstrap on instances (the natural unit, since claims within an instance share a profile/goal/model), 10,000 iterations, seed 42. We primarily report paired differences. For repair-effect analysis, pairs are matched by (profile, goal, model). For cross-model contrasts, pairs are matched by (profile, goal).

Baselines. We compare three operating points: (i) *verify+filter only* ($r=0$ in Table 2); (ii) *full VERITYGATE* ($r=1$: verify+filter+repair); (iii) *ungrounded narration* (high-level summary only, every “Ungrounded” row). A *no-defense* (ship-all, no filter) baseline is not run; the complement of the $r=0$ survival fraction quantifies the user-visible failure rate (e.g., 80.3% at MINI grounded $r=0$). We do not evaluate LLM-as-judge as a *filter* baseline: our judge study (Section 4.3) finds that GPT-4o prefers ungrounded narrations over verified ones and chooses slot A in 83% of calls. Because it sees no evidence graph, we do not treat it as a claim-level filter. Instead, the judge is a diagnostic on *whether automatic preference signals track faithfulness*: they do not.

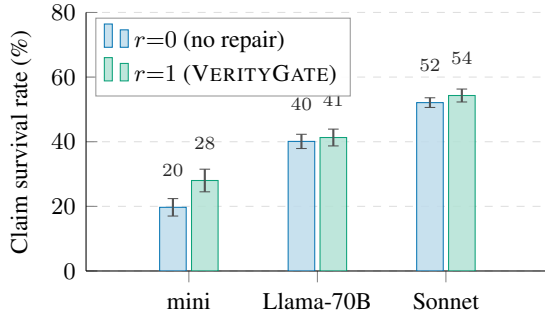


Figure 3: Claim-survival rate by model and repair setting, with 95% cluster-bootstrap CIs ($n=150$ per cell). Repair helps the smallest model most; Llama’s change is inconclusive; Sonnet gains marginally.

4.2 Main results

Table 2 reports per-cell claim-survival rate, shipped verified claims per instance, and four-gate breakdown at both $r=0$ and $r=1$. Across all three model tiers, the grounded variant under repair ships more verified claims than the ungrounded variant; the gap widens with model capability. On the frontier model, the grounded variant under repair ships 6.67 verified claims per instance.

Figure 3 visualizes the per-model survival rates across both repair settings, with bootstrap CI bars.

Per-model repair effect (paired, $r=1 - r=0$).

Table 3 shows the paired bootstrap differences. Repair helps small models a lot, frontier models a little, and the open-weights mid-tier not significantly.

Cross-model contrasts at $r=1$. The three-tier ordering is clean with non-overlapping CIs: Sonnet – mini = +26.4 pts [+22.6, +30.1], Llama – mini = +13.3 pts [+9.2, +17.4], and Sonnet – Llama = +13.0 pts [+10.3, +15.7].

Grounded vs ungrounded: rate is not volume.

On mini at $r=0$, the ungrounded variant has a *higher* per-claim survival rate (26.1%) than the grounded variant (19.7%) even though it ships *fewer* verified claims per instance (1.29 vs 1.60). Without an evidence graph the model produces a shorter, safer narration that references fewer entities and numbers, so each individual claim is less likely to fail any gate, but the user receives less verifiable content overall. The deployment-relevant metric is therefore shipped verified claims per instance, not survival rate alone, which is why we pre-registered both as co-primary.

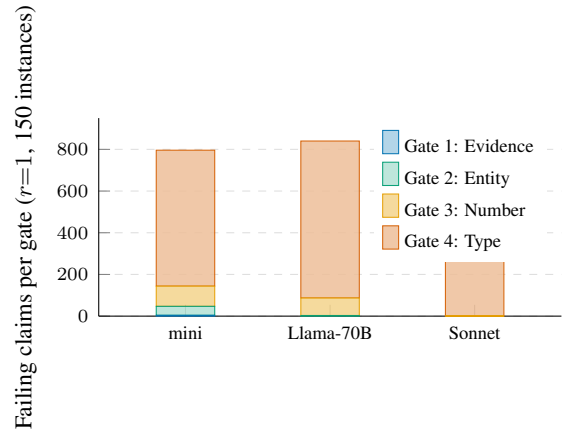


Figure 4: Failing claims per gate on the grounded variant at $r=1$ (150 instances per model; claims are counted once within each gate and may appear under multiple gates). Gate 4 (claim-type rules) dominates on every tier and is the residual problem after repair nearly eliminates Gates 1–3 on the frontier model. On Sonnet, all 841 distinct failing claims violate Gate 4; one also violates Gate 3. This is a persistent residual mode in the tested cells, not a universal scaling claim.

Diminishing returns and honesty under pressure.

A second repair pass on Sonnet gives $\Delta(r=2 - r=1) = +0.77$ pts [−1.22, +2.81] (CI crosses zero), so we deploy at $r=1$. Capable models become more conservative under repair: mean shipped count drops slightly on Llama ($\Delta = -0.71$) and Sonnet ($\Delta = -0.47$), abandoning claims they cannot substantiate rather than producing wrong ones. At $r=1$, Sonnet still ships 6.67 verified claims per instance, more than either other evaluated model; its own $r=0$ cell ships 7.15.

Where the failures live.

Figure 4 visualizes the gate-error breakdown on the grounded variant under one repair pass. Gate 4 (claim-type rules) dominates across all three tiers: it affects 652/672 (97.0%), 753/763 (98.7%), and 841/841 (100%) distinct failing claims on mini, Llama, and Sonnet, respectively. The sole Sonnet Gate 3 failure overlaps a Gate 4 failure. Gates 1–3 are essentially eliminated on Sonnet under repair (0 evidence errors, 0 entity errors, 1 number error across 150 instances). Within the tested models, the residual bottleneck is type categorization rather than allowlisting or number-grounding.

Structural-only verification: prose-vs-citation gap.

A small external rater study (Appendix A) surfaces two limitations distinct from Gate 4. On four items, all three raters accepted the prose val-

Model	Variant	Claim survival		Shipped / inst.		Failing claims per gate ($r=1$, total)			
		$r=0$	$r=1$	$r=0$	$r=1$	Evid.	Ent.	Num.	Type
GPT-4O-MINI	Grounded	19.7%	28.0%	1.60	1.74	4	43	97	652
	Ungrounded	26.1%	25.6%	1.29	1.31	4	27	263	486
LLAMA-3.3-70B	Grounded	40.1%	41.3%	4.29	3.58	0	3	84	753
	Ungrounded	21.6%	22.0%	0.99	1.02	0	344	334	227
CLAUDE SONNET 4.6	Grounded	52.1%	54.3%	7.15	6.67	0	0	1	841
	Ungrounded	43.2%	41.6%	2.36	2.21	0	46	196	294

Table 2: Main results for 900 benchmark instances at each repair setting (50 profiles $\times$ 3 goals $\times$ 3 narrators $\times$ 2 variants). Results are separate for $r=0$ and $r=1$. Conditions: *Grounded*/ $r=0$ = verify+filter, no repair pass; *Grounded*/ $r=1$ = full VERITYGATE (verify+filter+repair); *Ungrounded*/ r = same pipeline, ungrounded prompt. The $r=0$ and $r=1$ columns are per-pass raw aggregates; the deployment policy (Appendix E) is per-instance best-of selection. Claim survival = passing claims / parsed-and-retained claims. “Shipped / inst.” is mean verified claims per instance. The last four columns report **distinct failing claims per gate** at $r=1$ (once within each gate). A claim may appear in multiple gate columns, so their sum is not a distinct-claim total. Table 7 separately breaks Gate 4 into *verifier messages* by claim type.

Model	Δ survival	Sig.
GPT-4O-MINI	+8.3 pts [+4.3, +12.5]	yes
LLAMA-3.3-70B	+1.2 pts [-1.3, +3.6]	no
CLAUDE SONNET 4.6	+2.2 pts [+0.4, +4.1]	yes

Table 3: Paired Δ ($r=1 - r=0$) on grounded claim survival, 95% cluster bootstrap CI, paired by (profile, goal, model). Llama shows no detectable survival gain under one pass. Paired Δ on shipped verified claims per instance: mini +0.14 [−0.17, +0.47] (NS); Llama −0.71 [−1.05, −0.37] (down significantly, consistent with becoming more conservative under repair); Sonnet −0.47 [−0.86, −0.08] (down significantly).

ues while the verifier rejected the declared number strings, mainly because numerically equivalent forms such as 2400.0 and 2400.00 did not match exactly; one item also failed a Gate 4 rule. In the opposite direction, one unanimous NO vs verifier YES case contained an undeclared prose phrase and empty cited fields, leaving the verifier nothing to inspect. Together, these cases motivate combined *structural* + *prose* verification and explicit numeric canonicalization as future work.

4.3 Counterbalanced Judge Analysis

We additionally evaluate user-facing usefulness via a counterbalanced LLM-as-judge protocol. The judge sees two explanations of the same optimization result: the *verifier-emitted prose subset* of the grounded narration (i.e., the user-visible output after permissive emission) and the raw ungrounded narration, without knowing which is which and without access to the evidence graph. To control for position bias, every pair is judged twice with the order swapped.

A pair score is computed by averaging the two ordering-wise grounded scores, producing one of {1.0, 0.75, 0.5, 0.25, 0.0} plus an *inconsistent* flag for pairs where the judge flipped its verdict across orderings.

Across a deterministic stratified slice of 150 matched pairs (50 per narrator $\times$ 3 narrators, the first 150 of 450 in lexicographic (profileId, goal, modelId) order) = 300 judge calls, we observe:

- **0% tie / parse-failure rate.** Every call returned a definitive A or B vote with mean confidence 0.91.
- **83% observed position-A rate.** Across the 300 counterbalanced calls, the judge picks the explanation in slot A 83% of the time.
- **Mean preference score < 0.5 on all three narrators.** Sonnet 0.30, mini 0.41, Llama 0.42: the judge prefers the *ungrounded* narration across the board (a usefulness preference, not a faithfulness measurement; the judge sees no evidence graph).

The 83% call-level position-A vote rate and 66% pair-level flip rate measure different quantities and are not algebraically interchangeable. Together they show a strong position effect while leaving room for content-sensitive decisions. Within the 34% of pairs where the judge agrees across orderings, ungrounded wins decisively.

Figure 5 visualizes the position-bias diagnostic and per-model preference distribution side by side.

This is the strongest argument for deterministic verification. A frontier LLM-as-judge, given full freedom to compare two explanations, picks the

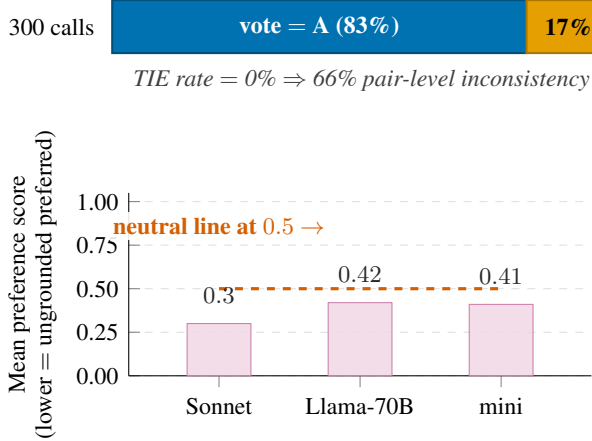


Figure 5: Counterbalanced judge diagnostic. *Top*: Position A is chosen in 83% of 300 calls and 66% of swapped pairs flip—a strong position effect, not proof of content independence. *Bottom*: All three mean scores are below 0.5, favoring raw ungrounded prose. Because the judge has no evidence graph, this is usefulness preference rather than faithfulness.

fluent ungrounded one (a preference signal, not faithfulness; no evidence access). Without counterbalancing, a one-shot judge would report “ungrounded wins 83 to 17”: a fake headline driven by position bias. Counterbalancing exposes the position effect and isolates the order-consistent subset; we do not interpret the remaining preference as content-independent.

External rater validation. On a 25-item Gates 1–3 sample, three independent raters reach 74% mean pairwise agreement and expose structural–prose mismatches. A separate 25-item Gate 4 study (auxiliary seed 43) finds 88% majority-of-three agreement with the strict rule. Appendices A–B give both protocols.

4.4 Cost and deployment characteristics

Cost estimates. Author-recorded estimates based on run token counts and vendor list prices on 2026-05-14 give per-explanation cost at $r=1$ of: GPT-4O-MINI \$0.001, LLAMA-3.3-70B \$0.0016, CLAUDE SONNET 4.6 \$0.05. Relative to Llama at $r=1$, Sonnet costs $\approx 31\times$ as much (\$0.05/\$0.0016) and ships $\approx 1.86\times$ as many verified claims per instance (6.67/3.58). The final estimates are archived, but the underlying provider token-count export was not preserved, so they are not independently re-estimable from the supplement.

Verifier microbench. On one CPU core using Java 21, a representative 14-claim instance takes

$p50/p95/p99 \approx 9/16/29 \mu s$ (10k warmup, 100k measured iterations), sustaining $\approx 88,000$ verifications/sec/core; observed post-run heap usage was 455.2 MB. LLM latency therefore dominates end-to-end walltime.

5 Discussion and Lessons Learned

Gate 4 isolated. On Sonnet at $r=1$, Gates 1–3 fall to 0/0/1 errors while Gate 4 has 841 distinct failing claims; a second pass has no clear effect. Across the tested tiers this suggests a type-routing bottleneck, not a universal ceiling.

Sparse-explanation edge case. At $r=1$ grounded, zero-verified-claim rates span **24.7%** (mini) to **1.3%** (Sonnet). Deployments need a minimum threshold, deterministic template fallback, or stronger narrator.

Generalization sketch. Graph builders and Gate 4 rules are domain-specific. Medical and loan-risk mappings are possible examples, not tested transfer.

6 Related Work

Structured-data and citation faithfulness. Slot-error methods check missing or spurious meaning-representation values (Wen et al., 2015; Dušek and Jurčiček, 2016; Juraska et al., 2018; Nie et al., 2019; Dušek et al., 2019); table-to-text metrics use extraction or reference alignment (Wiseman et al., 2017; Dhingra et al., 2019); and ALCE checks citations (Gao et al., 2023b). VERITYGATE adds a typed declaration/evidence contract and paired repair/filter evaluation, not a new set-membership operation (Appendix F).

Other defenses and metrics. NLI and atomic-fact metrics cover less structured evidence (Es et al., 2024; Rashkin et al., 2023; Min et al., 2023; Gao et al., 2023a; Yue et al., 2023; Chen et al., 2024). Complementary work studies constrained syntax (Willard and Louf, 2023), repair (Madaan et al., 2023; Yan et al., 2024; Huang et al., 2024), judge bias (Zheng et al., 2023; Wang et al., 2024), and tool safety (Radosevich and Halloran, 2025; Greshake et al., 2023).

Ethical Considerations

This work studies hallucination in LLM-generated explanations of structured decisions. All experiments were conducted in controlled local environments against a credit card portfolio benchmark.

No production systems or production end users were involved. Three external raters provided offline labels for a small sample of generated claims; the released rater files contain no names or directly identifying information. No directly identifying personal data was processed: the benchmark profiles used in this paper are generated parametrically; prior backend validation referenced in Section 4 used *public-use* BLS microdata, not personally identifiable records. We believe that documenting the failure modes of grounded LLM narration (and providing a deployable defense) serves the public interest, since organizations cannot protect against risks they do not understand. **Risk of false assurance from partial verification.** A “verified” label produced by VERITYGATE signals *schema-level* consistency between the declared claim fields and the evidence graph; it does *not* warrant that the prose shown to the user is faithful in full (see Appendix A). Deployments must surface this scope to end users (e.g., via product copy and/or audit logs) and must not present the label as a guarantee of complete factual accuracy. The [versioned GitHub code release](#) provides the framework implementation; the prompts, schemas, pre-registration, and reported per-cell logs are archived in the [versioned Zenodo supplementary artifact](#) under the repository’s permissive license.

Limitations

VERITYGATE has several limitations.

First, we evaluate against three LLM backends. While these span a wide capability range (small / open-weights mid / frontier), the rapidly evolving landscape means results may not generalize to all future models. The observed pattern (repair helps the small model most while Gate 4 remains dominant in all three tested tiers) may not hold for other models; both the pattern and absolute numbers require replication.

Second, we use a single seed (42) for profile generation. We report tight paired bootstrap CIs as the primary uncertainty quantification rather than running multiple seeds. This is sufficient for the within-bench claims but does not assess seed robustness.

Third, our external rater studies (Appendix A) both use small stratified samples (3 raters $\times$ 25 items each, for Gates 1–3 and for Gate 4 separately). Larger studies with domain-expert raters and broader coverage are a clear follow-up. Note

that the verifier’s primary check is against the deterministic backend’s allowlists, not against human judgment; the rater studies probe a different question (*do humans, given the same allowlists, agree with the verifier’s accept/reject calls?*) rather than re-deriving the ground truth.

Fourth, the LLM-as-judge analysis was conducted using a single judge model (GPT-4O). Other judge models may exhibit different position-bias magnitudes and different absolute preferences. We report the judge result as a secondary metric and explicitly tag it as “blinded pairwise usefulness preference under counterbalancing,” not as a faithfulness measure.

Fifth, adversarial robustness is not evaluated. The verifier is sound against non-adversarial LLM hallucination, but a model that knows the allowlist could craft outputs that pass verification while remaining semantically misleading. This is out of scope for this paper, since our threat model is unintentional fabrication, not adversarial attack.

Sixth, all empirical results are on a single application domain (credit-card portfolio optimization with the MILP backend described in §4). The verifier interface, claim schema, and Gate 4 type rules are designed to be portable, and we sketch mappings to medical decision support and loan-risk assessment in Section 5; neither mapping is empirically tested. Empirical cross-domain transfer is the most important follow-up.

Seventh, we do not compare narrative quality or coverage against a deterministic evidence-graph template, ablate strict versus relaxed Gate 4 rules, or run NLI/RAGAS/ALCE scorers on the generated outputs. The results therefore establish the behavior of the proposed structural audit-and-filter contract, not superiority over template generation or complementarity to neural faithfulness metrics.

References

- British Columbia Civil Resolution Tribunal. 2024. Mofatt v. air canada, 2024 bccrt 149. <https://www.canlii.org/en/bc/bccrt/doc/2024/2024bccrt149/2024bccrt149.html>. Decision dated February 14, 2024. Accessed May 2026.
- Damien Charlotin. 2025. AI hallucination cases database. <https://www.damiencharlotin.com/hallucinations/>. Database is updated continuously; figure cited (over 1,400 rulings) reflects a May 2026 snapshot. Accessed May 2026.

- Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. 2024. [Benchmarking large language models in retrieval-augmented generation](#). In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17754–17762.
- Bhuvan Dhingra, Manaal Faruqui, Ankur Parikh, Ming-Wei Chang, Dipanjan Das, and William Cohen. 2019. [Handling divergent reference texts when evaluating table-to-text generation](#). In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4884–4895, Florence, Italy. Association for Computational Linguistics.
- Ondřej Dušek, David M. Howcroft, and Verena Rieser. 2019. [Semantic noise matters for neural natural language generation](#). In *Proceedings of the 12th International Conference on Natural Language Generation*, pages 421–426, Tokyo, Japan. Association for Computational Linguistics.
- Ondřej Dušek and Filip Jurčiček. 2016. [Sequence-to-sequence generation for spoken dialogue via deep syntax trees and strings](#). In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 45–51, Berlin, Germany. Association for Computational Linguistics.
- Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. [RAGAs: Automated evaluation of retrieval augmented generation](#). In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, St. Julians, Malta. Association for Computational Linguistics.
- Alvan R. Feinstein and Domenic V. Cicchetti. 1990. [High agreement but low kappa: I. the problems of two paradoxes](#). *Journal of Clinical Epidemiology*, 43(6):543–549.
- Luyu Gao, Zhuyun Dai, Panupong Pasupat, Anthony Chen, Arun Tejasvi Chaganty, Yicheng Fan, Vincent Zhao, Ni Lao, Hongrae Lee, Da-Cheng Juan, and Kelvin Guu. 2023a. [RARR: Researching and revising what language models say, using language models](#). In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16477–16508, Toronto, Canada. Association for Computational Linguistics.
- Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023b. [Enabling large language models to generate text with citations](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6465–6488, Singapore. Association for Computational Linguistics.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. [Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection](#). In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security, AISEC ’23*, pages 79–90, New York, NY, USA. Association for Computing Machinery.
- Kilem L. Gwet. 2008. [Computing inter-rater reliability and its variance in the presence of high agreement](#). *British Journal of Mathematical and Statistical Psychology*, 61(1):29–48.
- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. [Large language models cannot self-correct reasoning yet](#). In *The Twelfth International Conference on Learning Representations*.
- Juraj Juraska, Panagiotis Karagiannis, Kevin Bowden, and Marilyn Walker. 2018. [A deep ensemble model with slot alignment for sequence-to-sequence natural language generation](#). In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 152–162, New Orleans, Louisiana. Association for Computational Linguistics.
- J. Richard Landis and Gary G. Koch. 1977. [The measurement of observer agreement for categorical data](#). *Biometrics*, 33(1):159–174.
- Colin Lecher. 2024. [NYC’s AI chatbot tells businesses to break the law](#). The Markup. Accessed May 2026.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. [Self-refine: Iterative refinement with self-feedback](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 46534–46594. Curran Associates, Inc.
- Sewon Min, Kalpesh Krishna, Xinxu Lyu, Mike Lewis, Wen-tau Yih, Pang Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. [FActScore: Fine-grained atomic evaluation of factual precision in long form text generation](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12076–12100, Singapore. Association for Computational Linguistics.
- Feng Nie, Jin-Ge Yao, Jinpeng Wang, Rong Pan, and Chin-Yew Lin. 2019. [A simple recipe towards reducing hallucination in neural surface realisation](#). In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2673–2679, Florence, Italy. Association for Computational Linguistics.
- Bernie Pazanowski. 2025. [Morgan & morgan lawyers fined for “hallucinated” AI citations](#). Bloomberg Law. *Wadsworth v. Walmart*, U.S. District Court for the District of Wyoming; Judge Kelly H. Rankin sanctions order dated February 24, 2025. Accessed May 2026.

Brandon Radosevich and John Halloran. 2025. *MCP safety audit: LLMs with the model context protocol allow major security exploits*. *arXiv preprint arXiv:2504.03767*.

Hannah Rashkin, Vitaly Nikolaev, Matthew Lamm, Lora Aroyo, Michael Collins, Dipanjan Das, Slav Petrov, Gaurav Singh Tomar, Iulia Turc, and David Reitter. 2023. *Measuring attribution in natural language generation models*. *Computational Linguistics*, 49(4):777–840.

U.S. Bureau of Labor Statistics. 2024. Consumer expenditure surveys (CE), interview survey: Public-use microdata (PUMD), 2020–2024. <https://www.bls.gov/cex/pumd.htm>. Accessed January 2026.

Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Lingpeng Kong, Qi Liu, Tianyu Liu, and Zhifang Sui. 2024. *Large language models are not fair evaluators*. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9440–9450, Bangkok, Thailand. Association for Computational Linguistics.

Tsung-Hsien Wen, Milica Gašić, Nikola Mrkšić, Pei-Hao Su, David Vandyke, and Steve Young. 2015. *Semantically conditioned LSTM-based natural language generation for spoken dialogue systems*. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 1711–1721, Lisbon, Portugal. Association for Computational Linguistics.

Brandon T. Willard and Rémi Louf. 2023. *Efficient guided generation for large language models*. *Preprint*, arXiv:2307.09702.

Sam Wiseman, Stuart Shieber, and Alexander Rush. 2017. *Challenges in data-to-document generation*. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 2253–2263, Copenhagen, Denmark. Association for Computational Linguistics.

Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. *Corrective retrieval augmented generation*. *arXiv preprint arXiv:2401.15884*.

Xiang Yue, Boshi Wang, Ziru Chen, Kai Zhang, Yu Su, and Huan Sun. 2023. *Automatic evaluation of attribution by large language models*. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 4615–4635, Singapore. Association for Computational Linguistics.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph Gonzalez, and Ion Stoica. 2023. *Judging LLM-as-a-judge with MT-Bench and chatbot arena*. In *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623. Curran Associates, Inc.

Appendix A. Independent Rater Validation

Three independent raters with computer science backgrounds (no specific ML training) each rated the same stratified 25-item sample drawn from our paper-grade evaluation run: 10 items the verifier classified as VERIFIED and 15 items rejected by Gates 1–3. Items rejected by Gate 4 were excluded from the rater protocol because that gate requires domain-specific structural judgment beyond the rubric’s scope; we discuss Gate 4 separately in Section 5.

Raters followed a fixed rubric (Appendix B) that asked a single binary question per item: *Does this claim reference only entities and numbers that appear in the evidence box?* The rubric was designed to require careful reading and table comprehension only, matching the mechanical nature of Gates 1–3. Raters had no visibility into the verifier’s classification when rating. Items rated UNCLEAR by any rater were excluded from agreement calculations following standard practice.

Mean pairwise agreement across the three raters was 74% on the 25-item sample. Table 4 reports the full chance-corrected agreement statistics: pairwise Cohen’s κ , Fleiss’ κ across all three raters, and each rater’s agreement with the verifier as the reference label. Fleiss’ $\kappa = 0.48$ indicates moderate inter-rater agreement (Landis and Koch, 1977); pairwise Cohen’s κ ranges from 0.35 (substantively fair) to 0.70 (substantial), with the wide span itself informative: rubric-following raters can disagree even on a deterministic structural check. Rater-vs-verifier κ is markedly lower (0.00–0.30), and the majority-of-3 vs. verifier κ is 0.02, consistent with the systematic structural-vs-prose disagreement pattern we discuss below. We interpret these numbers as supporting, not undermining, the case for mechanical verification: human raters disagree both with each other and (more sharply) with the verifier, while the verifier is reproducibly self-consistent across runs.

The qualitative disagreement pattern between raters and the verifier is systematic rather than noise:

- **All three raters disagreed with the verifier on four items** (a unanimous YES vs. verifier NO). The raters treated numerically equivalent trailing-zero forms as the same, while the verifier required exact membership of each declared number string. For example, 2400.0 did not match the allowed 2400.00; one of

Comparison	Agreement	Cohen's κ	n
Rater 1 vs Rater 2	69.6%	0.39	23
Rater 1 vs Rater 3	66.7%	0.35	21
Rater 2 vs Rater 3	85.7%	0.70	21
Rater 1 vs Verifier	65.2%	0.30	23
Rater 2 vs Verifier	52.0%	0.03	25
Rater 3 vs Verifier	42.9%	0.00	21
Majority-of-3 vs Verifier	50.0%	0.02	24
Fleiss' κ (3 raters)	—	0.48	21

Table 4: Chance-corrected inter-rater and rater-vs-verifier agreement on the 25-item rater sample. UNCLEAR votes dropped from each comparison; sample sizes reflect per-pair retention. Fleiss' κ uses items where all three raters voted YES/NO (21 items).

the four items also failed Gate 4. This exposes an exact-string canonicalization limitation.

- **All three raters disagreed with the verifier in the opposite direction on one item** (unanimous NO vs. verifier YES). The claim's prose mentioned a phrase not in the allowed-entity list ("general purchases"), but the LLM emitted empty cited fields and the verifier had nothing to check.

These five items (4 + 1) account for the majority of the gap between rater majority vote and the verifier. They argue for numeric canonicalization and combined *structural* + *prose-level* verification as future work, while the structural check alone catches the failure classes (Gates 1–4) with reproducibility that subjective human judgment partially lacks (3 raters do not perfectly agree among themselves; mean pairwise agreement 74%).

Walkthrough 1: Verified claim, unanimous agreement (item 4). Claim: "*chase-freedom-flex earns \$197.64, credits \$0.00, fee \$0.00, net \$197.64*". Allowed entities include `chase-freedom-flex`; allowed numbers include 0 and 197.64. Every entity and number in the prose appears in the evidence box. Verifier: YES. All three raters: YES. Clean case testing the rubric's *strict-literal* criterion.

Walkthrough 2: Gate 3 violation, all raters agree (item 13). Claim: "*The Capital One Savor card offers 4% cashback on dining, making it ideal for food-related expenses compared to standard cashback cards.*" Allowed entities include `cap1-savor` (matches "Capital One Savor" by paraphrase) and `DINING`; allowed numbers do not

include 4. Verifier: NO (Gate 3). All three raters: NO (correctly identified the unallowed "4"). A clean Gate 3 violation case where rater majority and verifier converge on rejection.

Walkthrough 3: Verifier-vs-rater disagreement, structural-only limitation (item 17).

Claim: "*Segment allocation for ONLINE shows amazon-prime-visa at \$2400.00 and amex-bcp at \$6000.00.*" Allowed entities include `ONLINE`, `amazon-prime-visa`, `amex-bcp`; allowed numbers include 2400 and 6000. Every prose element appears in the displayed evidence box by inspection. All three raters: YES. Verifier: NO. The declared numbers were 2400.0 and 6000.0, whereas the allowed strings were 2400.00 and 6000.00. The prose values are numerically correct, but the evaluated exact-string check rejects the structured fields. This case motivates canonical numeric comparison in a future verifier version.

Gate 4 validation sub-study. The 25-item study above by design covers Gates 1–3 only. Because Gate 4 dominates residual failures, we ran a separate three-rater sub-study over 25 stratified Gate 4 failures sampled from a re-instrumented smoke run (7 COMPARISON + 6 FEE_JUSTIFICATION + 6 CAP_SWITCH + 6 ASSUMPTION, round-robin across the three narrators, seed 43). Raters saw the claim text and the human-readable types of the evidence records the LLM cited; the verifier's expected-type column and decision were withheld to avoid priming. Raters judged whether the cited evidence type(s) were a reasonable backing for the claim text (AGREE) or not (DISAGREE); UNCLEAR was a third option.

Table 5 reports the agreement statistics. **Majority-of-3 raters agreed with the verifier's rejection on 22 of 25 items (88%);** mean per-rater agreement with the verifier was 86% (92%, 71%, 96%). Pairwise Cohen's κ ranges 0.19–0.36 and Fleiss' κ across the three raters is 0.19, but these chance-corrected values are noise-dominated under the highly skewed prevalence (most items are clearly verifier-correct rejections): when $p_e \approx p_o$, κ collapses irrespective of the substantive level of agreement (Feinstein and Cicchetti, 1990; Gwet, 2008). κ against the verifier reference is undefined by construction (the verifier rejected every item, giving zero variance) and is reported here as 0.00 for completeness. Raw % agreement is the substantive metric under this prevalence regime.

The single rater with the lowest verifier agree-

Comparison	Agreement	Cohen’s κ	n
Rater 1 vs Rater 2	79.2%	0.36	24
Rater 1 vs Rater 3	88.0%	−0.06	25
Rater 2 vs Rater 3	75.0%	0.19	24
Rater 1 vs Verifier	92.0%	0.00	25
Rater 2 vs Verifier	70.8%	0.00	24
Rater 3 vs Verifier	96.0%	0.00	25
Majority-of-3 vs Verifier	88.0%	0.00	25
Fleiss’ κ (3 raters)	—	0.19	24

Table 5: Gate 4 validation sub-study: rater agreement on 25 stratified Gate 4 failures, blind rubric (verifier’s expected type and decision hidden). κ values are deflated by the prevalence-skew paradox (verifier rejected every item, raters mostly agreed) and are reported alongside raw agreement, which is the substantive metric under skewed marginals. Majority-of-3 vs verifier 88% is the headline.

ment (Rater 2, 71%) showed a coherent pattern: they accepted `RESULT_BREAKDOWN` (final-money-totals records) as adequate backing for `COMPARISON` claims (e.g., “*compares result breakdown*”). This is a defensible *looser-grounding* interpretation in which related-but-not-exact evidence types are acceptable. We report it as a finding rather than noise: Gate 4 enforces a strict type-rule discipline, but a looser standard is intelligible and an alternative design choice. The fact that two raters land at $\geq 92\%$ verifier agreement and one rater at 71% with a coherent rationale gives a useful range, not a sharp validation ceiling.

Appendix B. Rater Rubric

Raters received the following one-page protocol prior to evaluation:

For each paired (claim, evidence) item, answer: *Does this claim reference only entities and numbers that appear in the evidence box?* Choose one of: **YES** (every entity and number in the claim appears in the evidence box), **NO** (at least one entity or number is not in the evidence box), or **UNCLEAR** (the claim is too vague to extract entities/numbers, or an entity name is paraphrased and you cannot tell if it matches).

The rubric included five calibration examples covering YES, NO due to entity violation, NO due to number violation, UNCLEAR, and NO due to a derived (sum-of-allowed) number. Raters were instructed to be literal (do not use outside knowledge), to not perform arithmetic (derived numbers must appear directly in the evidence), and to trust their first read (mark UNCLEAR rather than debate for more than 60 seconds).

Appendix C. Claim Schema and Gate 4 Type Rules

This appendix gives the six-field prompt/schema contract. The evaluated parser requires `claimId`, `claimType`, and `text`; it defaults missing or non-array citation fields to empty arrays. Gates 1–3 inspect those declared arrays, and Gate 4 applies Table 6. The type rules are specific to the credit-card domain.

Claim object (per-claim). The prompt asks the LLM to emit a JSON object with six fields:

```
{
  claimId: string
  claimType: enum (one of the
six in Table 6)
  text: string           // the
user-visible prose
  citedEvidenceIds: array of
string                 // Gate 1 input
  citedEntities: array of
string                 // Gate 2 input
  citedNumbers: array of string
// Gate 3 input
}
```

Gates 1–3 check, respectively, that every element of `citedEvidenceIds` is a node in the evidence graph, that every element of `citedEntities` is in the per-instance allowed-entity set, and that every element of `citedNumbers` exactly matches a string in the per-instance allowed-number set. Gate 4 applies *type-specific structural rules*: it requires that at least one citation in `citedEvidenceIds` resolve to an evidence node of a required type. The six claim types and their Gate 4 rules in our credit-card domain are:

Gate 4 failure modes in practice. Table 7 breaks down the residual Gate 4 errors on the grounded variant at $r=1$ by the claim type that triggered the violation. On every model, `COMPARISON` (missing `WINNER_BY_CATEGORY` citation) and `FEE_JUSTIFICATION` (missing `FEE_BREAK_EVEN` citation) are the largest two sub-modes. `CAP_SWITCH` is a smaller fraction in absolute count but distinctive in its sub-structure. Sonnet produces 220 verifier messages from 106 distinct `CAP_SWITCH` claims. At claim level, 89 (84%) omit `CAP_HIT`, 72 (68%) omit `ALLOCATION_SEGMENT`, and 55 (52%) omit both. The last figure directly supports the interpretation that the model does not consistently recognize that `CAP_SWITCH` requires both

Claim type	Required evidence type(s) at Gate 4
COMPARISON	must cite WINNER_BY_CATEGORY
ALLOCATION	(no Gate 4 requirement)
THRESHOLD	(no Gate 4 requirement)
ASSUMPTION	must cite ASSUMPTION
FEE_JUSTIFICATION	must cite FEE_BREAK_EVEN
CAP_SWITCH	must cite CAP_HIT; when present in the graph, must cite both CAP_HIT and ALLOCATION_SEGMENT

Table 6: Gate 4 (claim-type rules) per claim type. The “required evidence type” column lists the evidence-graph node type(s) that at least one element of `citedEvidenceIds` must resolve to. CAP_SWITCH is the only claim type requiring two distinct evidence types (one of each) when those types are present in the graph; the separate unconditional CAP_HIT path explains the duplicate message unit described below. The others require at most one. ALLOCATION and THRESHOLD have no Gate 4 rule; their declared entities and numbers are checked only by Gates 2 and 3.

evidence types. At message level, the 220 total comprises 89 messages from the unconditional required-type check, 59 additional CAP_HIT messages from a guarded two-type check, and 72 ALLOCATION_SEGMENT messages. Thus any third message on a claim reflects the two verifier paths for CAP_HIT, not a data or aggregation error. Constrained decoding over the claim-type schema is the natural next intervention.

Gate 4 is predominantly type-routing. A re-instrumented smoke replay (3 profiles $\times$ 3 goals $\times$ 3 narrators at $r=1$, with per-claim cited-evidence type capture) yielded 139 grounded Gate 4 failures with full citation records. **Within this 27-instance replay, 138/139 (99.3%) had non-empty `citedEvidenceIds`:** nearly every rejected claim cited something, but not the required type; one was true under-citation. The dominant mis-routing patterns were: FEE_JUSTIFICATION claims cited WINNER_BY_CATEGORY in 62.5% of their failures (25 / 40); COMPARISON cited FEE_BREAK_EVEN in 30.1% (22 / 73); ASSUMPTION cited WINNER_BY_CATEGORY in 55.6% (10 / 18). The required evidence type was available in the graph in the great majority of cases (verified by sibling claims of the same type passing Gate 4 within the same instance, e.g., a FEE_JUSTIFICATION that cor-

rectly cited FEE_BREAK_EVEN). The prompt provides the evidence graph with each node’s type in JSON but does not enumerate the type $\rightarrow$ required-evidence mapping; even the repair pass, which includes the exact rule in the verifier-error feedback (e.g., “*X* must cite *Y* evidence”), does not reliably resolve the routing error. Mechanical Gate 4 enforcement catches this class of LLM limitation structurally.

Claim type	mini	Llama	Sonnet
COMPARISON	371 (55.9%)	394 (48.3%)	324 (33.9%)
FEE_JUSTIFICATION	185 (27.9%)	154 (18.9%)	268 (28.1%)
CAP_SWITCH	21 (3.2%)	136 (16.7%)	220 (23.0%)
ASSUMPTION	87 (13.1%)	131 (16.1%)	143 (15.0%)
Total error msgs	664	815	955

Table 7: **Gate 4 verifier error messages** by claim type (grounded variant, $r=1$, 150 instances per model). The cell counts report verifier error *messages*, not distinct failing claims – this is the key distinction from Table 2, whose Gate 4 column reports one count per distinct failing claim and is authoritative for the headline totals. CAP_SWITCH can generate up to three messages per claim because CAP_HIT is checked on two verifier paths and ALLOCATION_SEGMENT on one. This is why the totals exceed the Table 2 Gate 4 counts (e.g., Sonnet: 955 messages from 841 failing claims). COMPARISON (missing WINNER_BY_CATEGORY) is the largest sub-mode on every model.

Appendix D. Prompt Templates and Sample Evidence Graph

This appendix gives the verbatim system-prompt skeletons used in our runs and a redacted example evidence graph snippet for one instance. The [versioned Zenodo supplementary artifact](#) contains the full prompt source and reported per-instance logs; the excerpts here expose the high-signal portions of the prompt-engineering choices in the paper.

Grounded narrator prompt (skeleton).

```
You must output ONLY a valid
JSON array of
claim objects. Cite only the
evidence IDs
listed below.
Each claim must have these
fields:
- claimId (string)
- claimType (one of:
COMPARISON, THRESHOLD,
ALLOCATION, ASSUMPTION,
FEE_JUSTIFICATION,
CAP_SWITCH)
```

```

- text (string)
- citedEvidenceIds (array of
strings)
- citedEntities (array of card
ids or
category names)
- citedNumbers (array of
number strings)
Output ONLY a JSON array. No
commentary.
Evidence IDs you may cite (exact
strings):
    {evidenceIdsLine}
For citedEntities, use ONLY
these exact
strings (card ids, category
names, currencies):
    {allowedEntitiesLine}
For citedNumbers, use ONLY these
exact strings
(do not round, do not reformat;
small ints
0-3 are also allowed):
    {allowedNumbersLine}
Structured evidence (for
reference):
    {evidenceJson}

```

Ungrounded narrator prompt (skeleton). The ungrounded variant differs only in what evidence is supplied:

```

Explain why this credit card
portfolio is
a good choice. Be specific.
Cite card names,
category earn rates, dollar
amounts, and
supporting reasons.
{same CLAIM_SCHEMA block as
grounded}
Portfolio summary:
    {goal, portfolio card ids,
primaryCurrency, netValueUsd,
totalEarnValueUsd,
totalCreditValueUsd,
totalFeesUsd}

```

The grounded variant gives the LLM the full evidence graph plus verbatim allowed-entity and allowed-number lists; the ungrounded variant gives only top-line breakdown numbers (net, earn, credits, fees) and the portfolio card list. Neither prompt instructs the LLM to “hallucinate” or “be careful”; the difference is entirely the evidence provided.

Repair prompt (skeleton). On a repair pass, the original grounded prompt is reused with up to ten specific verifier errors appended:

```

{original grounded prompt}
Your previous output failed the
claim verifier
with these errors:
- citedEntity not allowed:

```

```

amex-gold
- citedNumber not in allowed
set: 4
- claim type COMPARISON
requires citation
of WINNER_BY_CATEGORY
evidence
(up to 10 errors shown)
Produce a corrected JSON array.
Cite only
evidence ids, entities, and
numbers listed
in the prompt above. Use
exactly the strings
provided.

```

Judge prompt (skeleton, used in §4.3).

```

You are an impartial judge
comparing two
explanations of a credit card
portfolio
recommendation. Choose the
explanation that
is most useful to a user reading
it.
Consider:
- Faithfulness to the context
- Clarity
- Helpfulness
Do not let length or position
influence you.
Output ONLY a single JSON object
on one line.
Schema: {"winner": "A" | "B" |
"TIE",
"rationale": "<one short
sentence>",
"confidence": <float 0.0 to
1.0>}
Context (the optimizer's actual
decision):
    {contextSummary: goal +
portfolio
ids only; no evidence graph}
Explanation A: {prose A}
Explanation B: {prose B}

```

Note that the judge sees *only the prose* of the two explanations plus the (goal, portfolio) header. It has no access to the evidence graph, the verifier output, or which explanation is grounded; this is why we report the judge result as a usefulness-preference diagnostic rather than a faithfulness measurement.

Sample evidence graph excerpt. A representative grounded instance emits an evidence graph with the following shape (entity ids replaced with public-domain placeholders for the excerpt):

```

{
  graphDigest:
    "sha256:5af1...",
  nodes: [
    {id:"ev_1",
type:"ALLOCATION_SEGMENT",

```

```

        card:
          "chase-freedom-flex",
          category:"DINING",
        spend:"1200.00",
        {id:"ev_2",
        type:"WINNER_BY_CATEGORY",
          category:"DINING",
          winner:
            "chase-freedom-flex",
            runnerUp:"amex-gold"},
        {id:"ev_3",
        type:"FEE_BREAK_EVEN",
          card:

"chase-sapphire-preferred",
          fee:"95.00",
        breakEven:"1267.00"},
        ... (typical instance has
        14-22 nodes)
      ],
      allowedEntities:
      [chase-freedom-flex,
        chase-sapphire-preferred,
        DINING,
        GROCERIES, TRAVEL, ... (30
        strings)],
      allowedNumbers: [0, 1, 2, 3,
        95.00, 197.64,
        1031.00, 1200.00, 1267.00,
        ...
        (typically 80-110
        strings)]
    }

```

The graph digest is computed by canonicalizing the node list (sorted by id, fields in fixed order, all numbers as decimal strings) and hashing with SHA-256. The verifier’s accept/reject decision is a deterministic function of (claim, graph) and is reproducible across runs with bit-identical input.

Appendix E. Pre-Registration Summary

The released `protocol/eval_preregistration.md` separates these pre-run decisions from later analysis and deployment choices:

- **Co-primary metrics:** (a) *claim survival* = passing / totalClaims per cell, where the implemented totalClaims counts parsed-and-retained claims; (b) mean shipped verified claims per instance under permissive emission.
- **Pairing units.** Repair-effect contrasts: paired by (profileId, goal, modelId). Cross-model contrasts at $r=1$: paired by (profileId, goal). Sonnet $r=2$ ablation: paired by (profileId, goal).
- **Statistical procedure.** Instance-cluster bootstrap (10,000 iterations, seed 42); report

paired differences with 95% CIs (significant iff excluding zero).

- **Run and decision rules.** Compare $r=0$ with $r=1$; claim a repair benefit only if a paired survival CI excludes zero. The pre-registered diminishing-returns rule used the Sonnet smoke $r=2 - r=1$ CI, which crossed zero at $N=9$.
- **Stop conditions.** Abort a run above 5% provider-call failures or 50% above its cost budget; diagnose before rerunning the affected cell.
- **Explicit non-claims** committed up-front:
 - No claim of state-of-the-art on any open-domain factuality / RAG benchmark.
 - No claim of empirical cross-domain transfer or seed robustness.
 - No unqualified claim that grounding beats the ungrounded variant.

Post-registration analysis and deployment choices. The per-instance best-of- $(r=0, r=1)$ deployment policy, the first-150 lexicographic judge sample, and the large-run Sonnet $r=2$ ablation were specified later. Table 2 and the main figures instead report $r=0$ and $r=1$ separately. The observed 1.78% schema-failed instance rate at $r=1$ is descriptive; the 5% stop rule concerned provider calls. The judge is a usefulness diagnostic, and permissive emission is a filtering policy, not evidence that the generator improves.

Appendix F. Structured-Data and Citation-Metric Lineage

Slot-error and semantic-control work checks missing or spurious meaning-representation values (Wen et al., 2015; Dušek and Jurčiček, 2016; Juraska et al., 2018; Nie et al., 2019; Dušek et al., 2019); IE and PARENT evaluate table-to-text content (Wiseman et al., 2017; Dhingra et al., 2019), while ALCE separates citation correctness and completeness (Gao et al., 2023b). VERITY-GATE adds a typed declaration/evidence contract plus repair-and-filter evaluation, not a new set-membership operation. Neural-metric and relaxed-rule comparisons remain future work.